

APPLIED STATISTICS, PROBABILITY THEORY AND MATHEMATICAL STATISTICS

Attila Barta, Sándor Pecsora

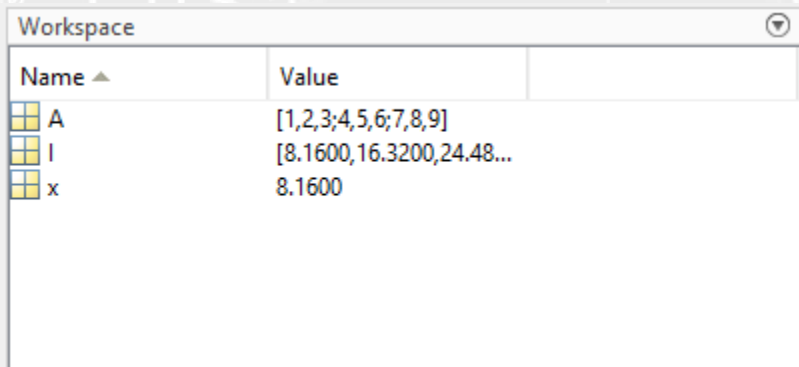
Introduction to Matlab

Variables in Matlab

Valid Names - A valid variable name starts with a letter, followed by letters, digits, or underscores. MATLAB® is case sensitive, so *A* and *a* are not the same variable. The maximum length of a variable name is the value that the `namelengthmax` command returns. You cannot define variables with the same names as MATLAB keywords, such as `if` or `end`. For a complete list, run the `iskeyword` command.

Create Variables - To create a new variable, enter the variable name in the Command Window, followed by an equal sign (=) and the value you want to assign to the variable. For example, if you run these statements, MATLAB adds the three variables *x*, *A*, and *I* to the workspace:

```
x = 8.16;  
A = [1 2 3; 4 5 6; 7 8 9];  
I = x*A;
```



The screenshot shows the MATLAB Workspace window with a table of variables. The table has two columns: 'Name' and 'Value'. There are three rows of data, each preceded by a small icon representing the variable's type (matrix for A and I, scalar for x).

Name	Value
A	[1,2,3;4,5,6;7,8,9]
I	[8.1600,16.3200,24.48...
x	8.1600

You do not have to declare variables before assigning values to them. If you do not end the assignment statement with a semicolon (;), MATLAB displays the result in the Command Window.

Scalar arithmetic operations

Symbol	Operation	Matlab form
$\wedge$	Exponentiation: a^b	a^b
$*$	Multiplication: ab	$a*b$
$/$	Right division: $a/b = \frac{a}{b}$	a/b
$\backslash$	Left division: $a \backslash b = \frac{b}{a}$	$a \backslash b$
$+$	Addition: $a + b$	$a+b$
$-$	Subtraction: $a - b$	$a-b$

An Example Session

$$8/10$$

ans =

$$0.8000$$

$$r=8/10$$

r =

$$0.8000$$

r

r =

$$0.8000$$

$$s=20*r$$

s =

$$16$$

Order of precedence

Precedence	Operation
First	Parentheses, evaluated starting with the innermost pair.
Second	Exponentiation, evaluated from left to right.
Third	Multiplication and division with equal precedence, evaluated from left to right.
Fourth	Addition and subtraction with equal precedence, evaluated from left to right.

Examples of Precedence

- $8 + 3*5$
- ans =
- 23
- $8 + (3*5)$
- ans =
- 23
- $(8 + 3)*5$
- ans =
- 55
- $4^{21}28/4*2$
- ans =
- 0

- $3*4^2 + 5$
- ans =
- 53
- $(3*4)^2 + 5$
- ans =
- 149
- $27^{(1/3)} + 32^{(0.2)}$
- ans =
- 5
- $27^{(1/3)} + 32^{0.2}$
- ans =
- 5

Commands for managing the work session

Command	Description
clc	Clears the Command window
clear	Removes all variables from memory
clear var1 var2	Removes variables var1 and var2 from memory
exist('name')	Determines if a file or variable exists having the name 'name'

Numeric display formats

Command	Description
format SHORT	Scaled fixed point format with 5 digits.
format LONG	Scaled fixed point format with 15 digits for double and 7 digits for single.
format SHORTE	Floating point format with 5 digits
format LONGE	Floating point format with 15 digits for double and 7 digits for single.
format SHORTG	Best of fixed or floating point format with 5 digits.
format LONGG	Best of fixed or floating point format with 15 digits for double and 7 digits for single.

Arrays

The numbers 0, 0.1, 0.2, ..., 10 can be assigned to the variable u by typing

```
u = 0:0.1:10.
```

To compute $w = 5 \sin u$ for $u = 0, 0.1, 0.2, \dots, 10$, the session is:

```
u = 0:0.1:10;
```

```
w = 5*sin(u);
```

The single line, $w = 5 \sin(u)$, computed the formula $w = 5 \sin u$ 101 times.

Array Index

`u(7)`

`ans =`

`0.6000`

`w(7)`

`ans =`

`2.8232`

Use the `length` function to determine how many values are in an array.

`m = length(w)`

`m =`

`101`

Some commonly used mathematical functions

Function	Matlab syntax
e^x	exp(x)
$\sqrt{x}$	sqrt(x)
$\ln x$	log(x)
$\log_{10} x$	log10(x)
$\cos x$	cos(x)
$\sin x$	sin(x)
$\tan x$	tan(x)
$\cos^{-1} x$	acos(x)
$\sin^{-1} x$	asin(x)
$\tan^{-1} x$	atan(x)
Important! The Matlab trigonometric functions use radian measure.	

Some Built-in functions

Function	Description
<code>mean(A)</code>	Average or mean value of A
<code>max(A), min (A)</code>	If A is a vector, then <code>min(A)</code> returns the smallest element of A. If A is a matrix, then <code>min(A)</code> is a row vector containing the minimum value of each column. <code>max(A)</code> works similarly, but with the maximum of A.
<code>sum(A)</code>	Summation of the elements of A
<code>sort(A)</code>	Sort in ascending or descending order.
<code>median(A)</code>	For vectors, <code>median(a)</code> is the median value of the elements in x. For matrices, <code>median(A)</code> is a row vector containing the median value of each column.
<code>std(A)</code>	Standard deviation
<code>det(A)</code>	Determinant of a square matrix
<code>inv(A)</code>	Inverse of the square matrix A
<code>size(A)</code>	<code>D=size(A)</code> , for M-by-N matrix A, returns the two-element row vector <code>D=[M,N]</code> containing the number of rows and columns in the matrix.
<code>length(A)</code>	Returns the length of vector A. It is equivalent to <code>MAX(SIZE(A))</code> for non-empty arrays and 0 for empty ones.
<code>flip(A)</code>	Flip the order of elements
<code>prod(A)</code>	<code>P=prod(A)</code> is the product of the elements of the vector A. If A is a matrix, P is a row vector with the product over each column.
Important! If A is a matrix, then some of these functions work differently, please check the helper.	

Operators (relational, logical)

Operator	Description
==	Equal to
~=	Not equal to
<	Strictly smaller
>	Strictly greater
<=	Smaller than or equal to
>=	Greater than or equal to
&	And operator
	Or operator

Vectors (One dimensional arrays)

To create a *row* vector, separate the elements by commas or spaces. Use square brackets:

```
p = [3, 7, 9]
```

```
p =  
    3    7    9
```

You can create a *column* vector by using the *transpose* notation (').

```
p = [3, 7, 9]'
```

```
p =  
    3  
    7  
    9
```

You can also create a column vector by separating the elements by semicolons. For example,

```
g = [3; 7; 9]
```

```
g =  
    3  
    7  
    9
```

You can create row vectors by "appending" one vector to another.

For example, to create the row vector u whose first three columns contain the values of

$$r = [2, 4, 20]$$

and whose fourth, fifth, and sixth columns contain the values of

$$w = [9, -6, 3]$$

you type

$$u = [r, w]$$

The result is the vector

$$u = [2, 4, 20, 9, -6, 3]$$

You can create column vectors by "appending" one vector to another.

For example, to create the column vector u whose first three rows contain the values of

$$r = [2; 4; 20]$$

and whose fourth, fifth, and sixth rows contain the values of

$$w = [9; -6; 3]$$

you type

$$u = [r; w]$$

The result is the vector

$$u = [2; 4; 20; 9; -6; 3]$$

The colon operator (:)

The colon operator (:) easily generates a large vector of regularly spaced elements.

Typing

```
x = m:q:n
```

or

```
x = (m:q:n)
```

creates a vector x of values with a spacing q . The first value is m . The last value is n if $m-n$ is an integer multiple of q . If not, the last value is less than n .

For example, typing $x = 0:2:8$ creates the vector $x = [0, 2, 4, 6, 8]$, whereas typing $x = 0:2:7$ creates the vector $x = [0, 2, 4, 6]$.

To create a row vector z consisting of the values from 5 to 8 in steps of 0.1, type

```
z = 5:0.1:8
```

If the increment q is omitted, it is presumed to be 1. Thus typing $y = -3:2$ produces the vector $y = [-3, -2, -1, 0, 1, 2]$.

Creating vectors with the `linspace` command

The `linspace` command also creates a linearly spaced row vector, but instead you specify the number of values rather than the increment.

The syntax is `linspace(x1, x2, n)`, where `x1` and `x2` are the lower and upper limits and `n` is the number of points.

For example, `linspace(5, 8, 31)` is equivalent to `5:0.1:8`.

`linspace` Linearly spaced vector.

`linspace(X1, X2)` generates a row vector of 100 linearly equally spaced points between `X1` and `X2`.

`linspace(X1, X2, N)` generates `N` points between `X1` and `X2`.

For `N = 1`, `linspace` returns `X2`.

Matrices (Two dimensional arrays)

If the matrix is small you can type it row by row, separating the *elements* in a given row with *spaces* or *commas* and separating the *rows* with semicolons. For example, typing

```
A = [2,4,10;16,3,7];
```

creates the following matrix:

$$A = \begin{bmatrix} 2 & 4 & 10 \\ 16 & 3 & 7 \end{bmatrix}$$

Remember, spaces or commas separate elements in different *columns*, whereas semicolons separate elements in different *rows*.

Creating Matrices from Vectors

Suppose $a = [1, 3, 5]$ and $b = [7, 9, 11]$ (row vectors). Note the difference between the results given by $[a \ b]$ and $[a;b]$ in the following session:

```
c = [a b];
```

```
c =
```

```
1 3 5 7 9 11
```

```
D = [a;b]
```

```
D =
```

```
1 3 5
```

```
7 9 11
```

Array Addressing

The colon operator selects individual elements, rows, columns, or "subarrays" of arrays. Here are some examples:

- $v(:)$ represents all the row or column elements of the vector v .
- $v(2:5)$ represents the second through fifth elements; that is $v(2)$, $v(3)$, $v(4)$, $v(5)$.
- $A(:,3)$ denotes all the elements in the third column of the matrix A .
- $A(:,2:5)$ denotes all the elements in the second through fifth columns of A .
- $A(2:3,1:3)$ denotes all the elements in the second and third rows that are also in the first through third columns.
- $v = A(:)$ creates a vector v consisting of all the columns of A stacked from first to last.
- $A(\text{end},:)$ denotes the last row in A , and $A(:,\text{end})$ denotes the last column.

Arrays

You can use array indices to extract a smaller array from another array. For example, if you first create the array **B**

$$B = \begin{bmatrix} 2 & 4 & 10 \\ 16 & 3 & 7 \\ 8 & 4 & 9 \end{bmatrix}$$

then type `C = B(2:3, 1:3)`, you can produce the following array:

$$C = \begin{bmatrix} 16 & 3 & 7 \\ 8 & 4 & 9 \end{bmatrix}$$

Array Addition and Subtraction

$$\begin{bmatrix} 6 & -2 \\ 10 & 3 \end{bmatrix} + \begin{bmatrix} 9 & 8 \\ -12 & 14 \end{bmatrix} = \begin{bmatrix} 15 & 6 \\ -2 & 17 \end{bmatrix}$$

The addition shown in the equation above is performed in MATLAB as follows:

```
A = [6,-2;10,3];
```

```
B = [9,8;-12,14]
```

```
A+B
```

Array subtraction is performed in a similar way.

```
A = [6,-2;10,3];
```

```
B = [9,8;-12,14]
```

```
A-B
```

Multiplication

Multiplying a matrix **A** by a scalar w produces a matrix whose elements are the elements of **A** multiplied by w .

$$3 \begin{bmatrix} 2 & 9 \\ 5 & -7 \end{bmatrix} = \begin{bmatrix} 6 & 27 \\ 15 & -21 \end{bmatrix}$$

This multiplication is performed in MATLAB as follows:

```
A = [2, 9; 5, -7];
```

```
3*A
```

```
ans =  
     6     27  
    15    -21
```

Multiplication of two *arrays*

MATLAB uses two definitions of multiplication:

(1) *array* multiplication (also called *element-by-element* multiplication), and

(2) *matrix* multiplication.

Multiplication of two *arrays*

Element-by-element multiplication is performed in MATLAB as follows:

```
A = [6,-2;10,3];
```

```
B = [9,8;-12,14]
```

```
A.*B
```

```
ans =
```

```
54    -16
```

```
-120   42
```

Matrix multiplication is performed in MATLAB as follows:

```
A = [6,-2;10,3];
```

```
B = [9,8;-12,14]
```

```
A*B
```

```
ans =
```

```
78    20
```

```
54   122
```

Element-by-element operations

Symbol	Operation	Form	Examples
+	Scalar-array addition	$A+b$	$[6,3]+2=[8,5]$
-	Scalar-array subtraction	$A-b$	$[8,3]-5=[3,-2]$
+	Array addition	$A+B$	$[6,5]+[4,8]=[10,13]$
-	Array subtraction	$A-B$	$[6,5]-[4,8]=[2,-3]$
.*	Array multiplication	$A.*B$	$[3,5].*[4,8]=[12,40]$
./	Array right division	$A./B$	$[2,5]./[4,8]=[2/4,5/8]$
.\	Array left division	$A.\backslash B$	$[2,5].\backslash[4,8]=[2\backslash 4,5\backslash 8]$
.^	Array exponentiation	$A.^B$	$2.^[3,5]=[2^3,2^5]$ $[3,5].^[2,4]=[3^2,5^4]$

Matrix-Matrix Multiplication

In the product of two matrices **AB**, the number of *columns* in **A** must equal the number of *rows* in **B**. The product **AB** has the same number of *rows* as **A** and the same number of *columns* as **B**. For example,

$$\begin{bmatrix} 6 & -2 \\ 10 & 3 \\ 4 & 7 \end{bmatrix} \begin{bmatrix} 9 & 8 \\ -5 & 12 \end{bmatrix} = \begin{bmatrix} 6 * 9 + (-2) * (-5) & 5 * 8 + (-2) * 12 \\ 10 * 9 + 3 * (-5) & 10 * 8 + 3 * 12 \\ 4 * 9 + 7 * (-5) & 4 * 8 + 7 * 12 \end{bmatrix} = \begin{bmatrix} 64 & 24 \\ 75 & 116 \\ 1 & 116 \end{bmatrix}$$

A = [6, -2; 10, 3; 4, 7];

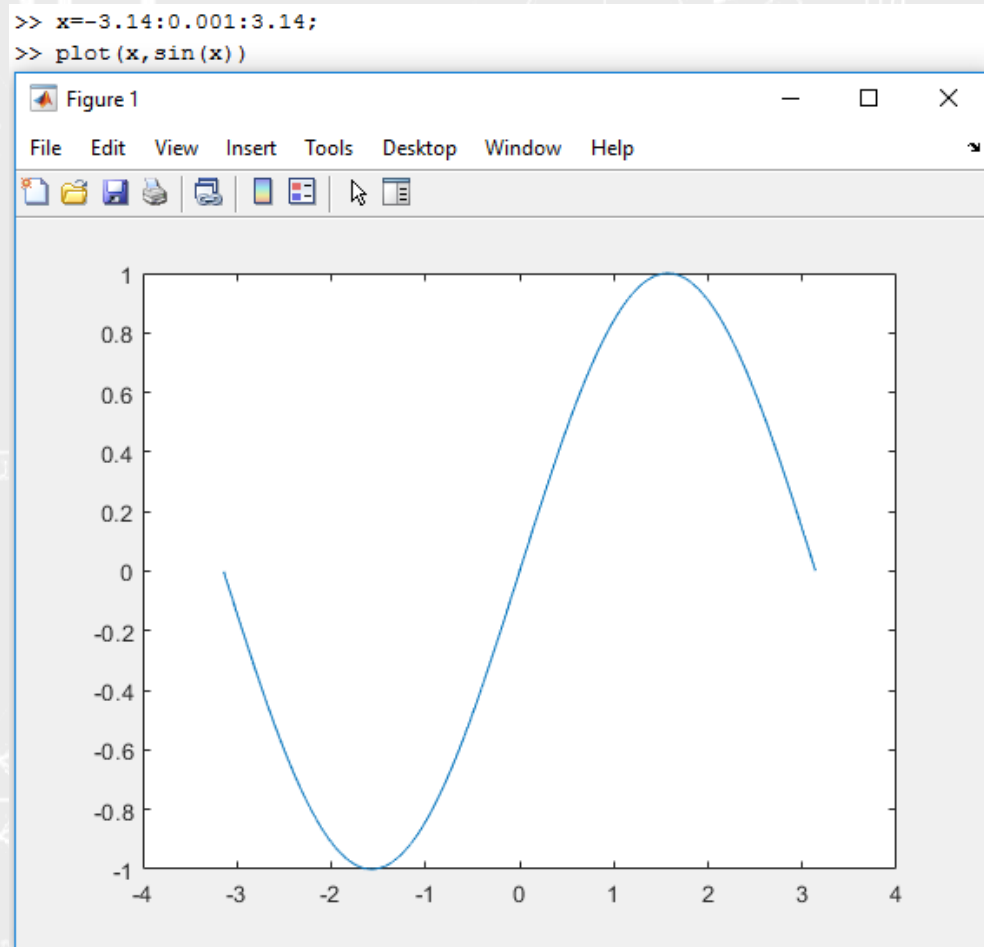
B = [9, 8; -5, 12];

A*B

ans =

64	24
75	116
1	116

A graphics window showing a plot



plot command in MATLAB

`plot(X,Y)` creates a 2-D line plot of the data in Y versus the corresponding values in X.

Example:

`X=[1,2,3,4,5]`

`X =`

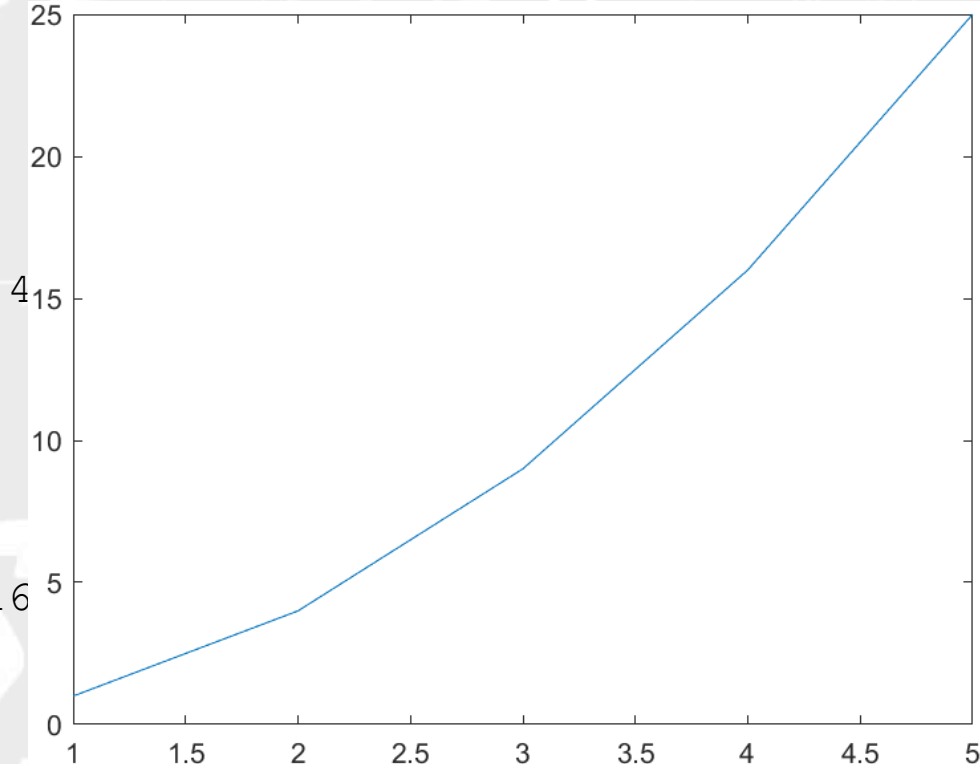
1 2 3

`Y=[1,4,9,16,25]`

`Y =`

1 4 9

`plot(X,Y)`



plot command in MATLAB

To plot the 5 point defined by X and Y, you can use for example the * marker.

Example:

```
X=[1,2,3,4,5]
```

```
X =
```

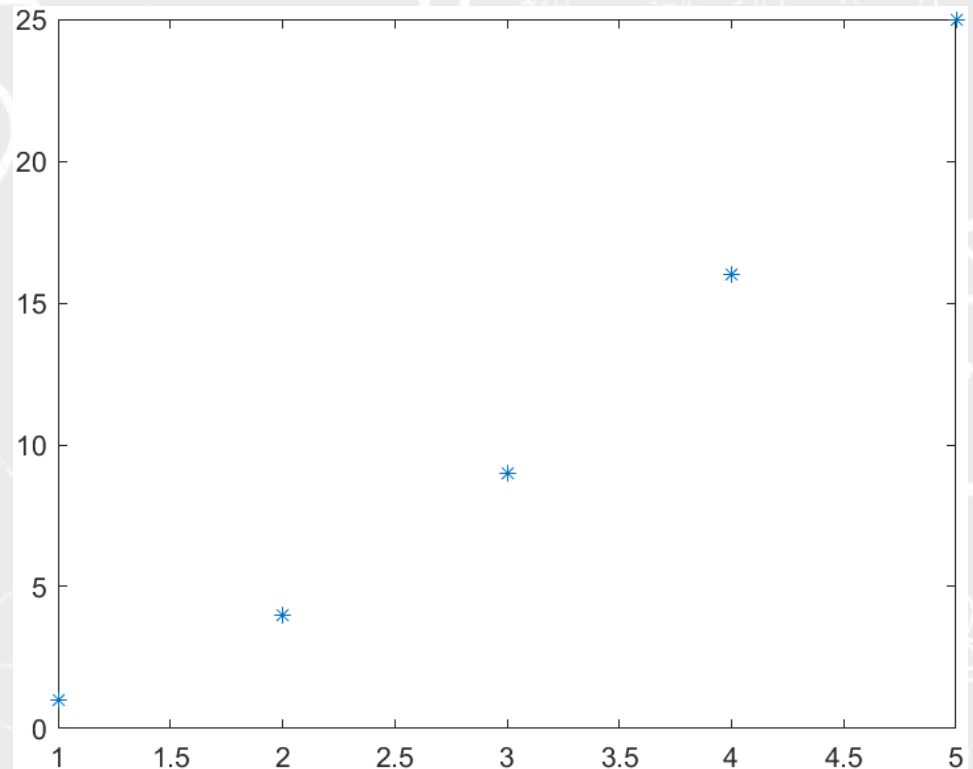
```
1     2     3     4     5
```

```
Y=[1,4,9,16,25]
```

```
Y =
```

```
1     4     9    16    25
```

```
plot(X,Y,'*')
```



plot line style, marker and colour

Line Style	Description
-	Solid line (default)
--	Dashed line
:	Dotted line
-.	Dash-dot line

Colour	Description
y	yellow
m	magenta
c	cyan
r	red
g	green
b	blue
w	white
k	black

Marker	Description
o	Circle
+	Plus sign
*	Asterisk
.	Point
x	Cross
s	Square
d	Diamond
^	Upward-pointing triangle
v	Downward-pointing triangle
>	Right-pointing triangle
<	Left-pointing triangle
p	Pentagram
h	Hexagram

Plotting Polynomials

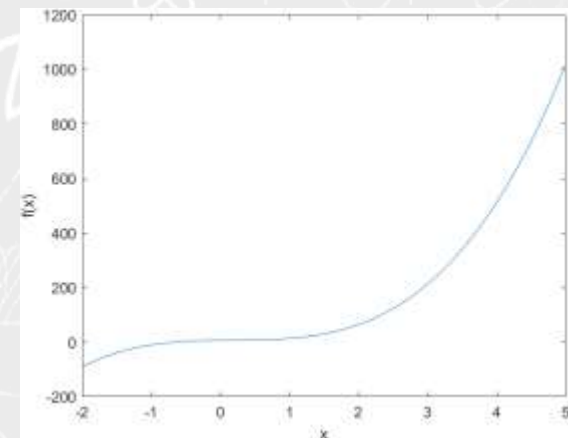
The function `polyval(a,x)` evaluates a polynomial at specified values of its independent variable `x`, which can be a matrix or a vector. The polynomial's coefficients of descending powers are stored in the array `a`. The result is the same size as `x`.

Example of Plotting a Polynomial

To plot the polynomial $f(x) = 9x^3 - 5x^2 + 3x + 7$

for $-2 \leq x \leq 5$, you type

```
a = [9,-5,3,7];  
x = -2:0.01:5;  
f = polyval(a,x);  
plot(x,f),xlabel('x'),ylabel('f(x)')
```



Plotting and labelling

The following MATLAB session plots $y = 0.4\sqrt{1.8x}$ for $0 \leq x \leq 52$, where y represents the height of a rocket after launch, in miles, and x is the horizontal (downrange) distance in miles.

```
x = 0:0.1:52;
```

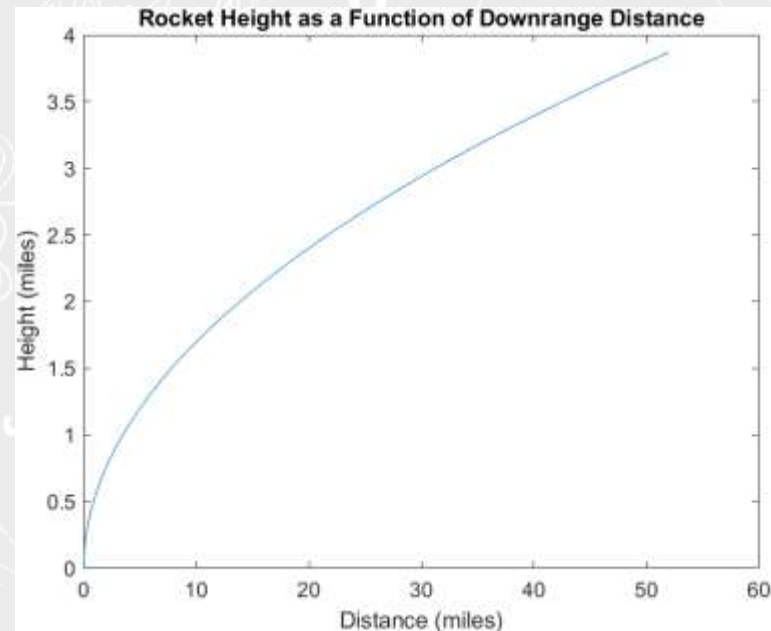
```
y = 0.4*sqrt(1.8*x);
```

```
plot(x,y)
```

```
xlabel('Distance (miles)')
```

```
ylabel('Height (miles)')
```

```
title('Rocket Height as a Function of Downrange Distance')
```



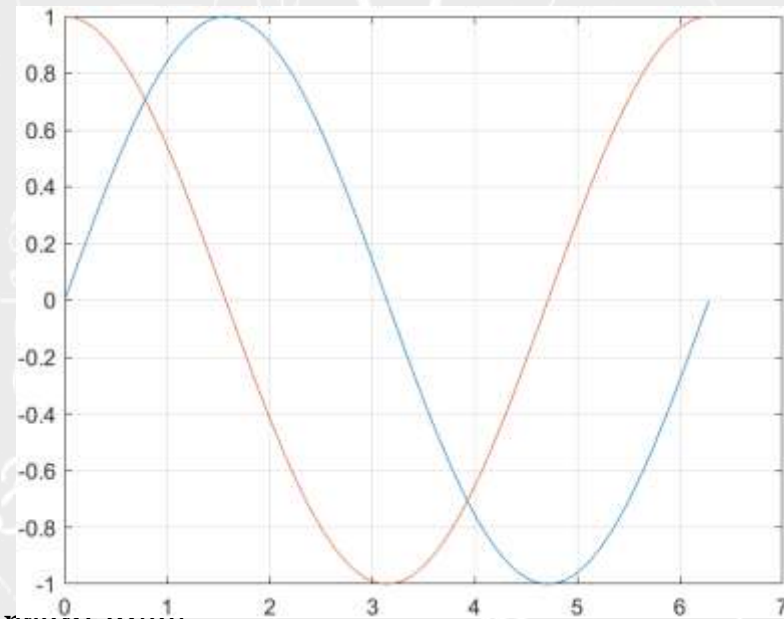
MULTIPLE GRAPHS

First method:

```
t = 0:pi/100:2*pi;  
y1=sin(t);  
y2=sin(t+pi/2);  
plot(t,y1,t,y2)  
grid on
```

grid on adds major grid lines to the current axes.

grid off removes major and minor grid lines from the current axes.



MULTIPLE GRAPHS

Second method (using the `hold on` command):

```
t = 0:pi/100:2*pi;
```

```
y1=sin(t);
```

```
y2=sin(t+pi/2);
```

```
plot(t,y1)
```

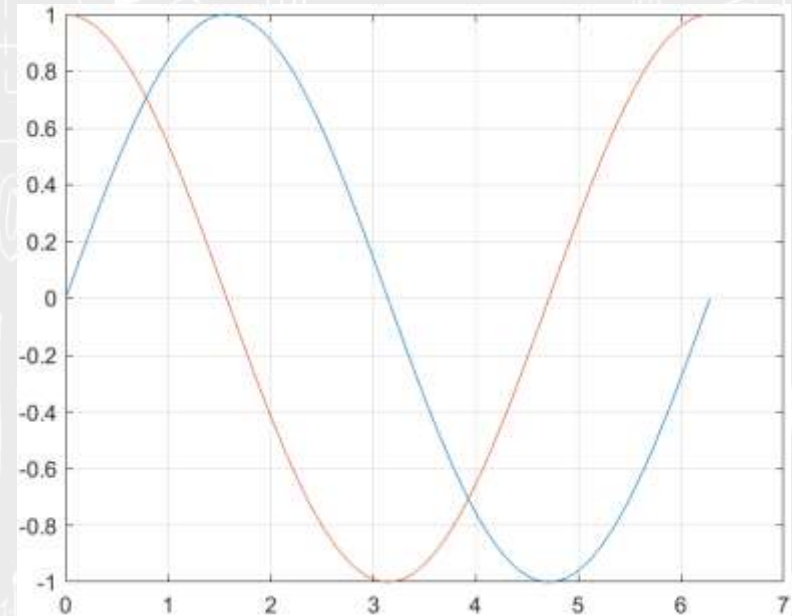
```
hold on
```

```
plot(t,y2)
```

```
grid on
```

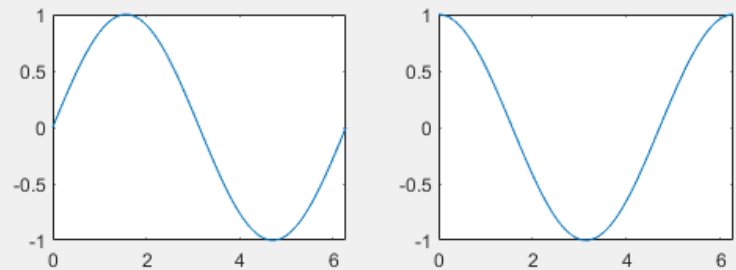
`hold on` holds the current plot and all axis properties, including the current color and linestyle, so that subsequent graphing commands add to the existing graph without resetting the color and linestyle.

`hold off` returns to the default mode whereby `PLOT` commands erase the previous plots and reset all axis properties before drawing new plots.



MULTIPLE PLOTS

```
t = 0:pi/100:2*pi;  
y1=sin(t);  
y2=sin(t+pi/2);  
subplot(2,2,1)  
plot(t,y1)  
subplot(2,2,2)  
plot(t,y2)
```



Graph Functions (summary)

Function	Description
plot	linear plot
stem	discrete plot
grid	add grid lines
xlabel	add X-axis label
ylabel	add Y-axis label
title	add graph title
subplot	divide figure window
figure	create new figure window
pause	wait for user response

You can perform operations in MATLAB in two ways

1. In the interactive mode, in which all commands are entered directly in the Command window.
2. By running a MATLAB program stored in *script* file. This type of file contains MATLAB commands, so running it is equivalent to typing all the commands—one at a time—at the Command window prompt. You can run the file by typing its name at the Command window prompt.

Scripts

The simplest type of MATLAB® program is called a *script*. A script is a file that contains multiple sequential lines of MATLAB commands and function calls. You can run a script by typing its name at the command line.

First click on the "New Script" icon:



Then the editor window appears, where you can edit your script:



My first scripts

In the "Editor" window you can write your script and save it on the local computer. (When saving the script do not forget about the naming regulations!)

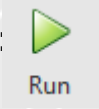
Example for a simple script:

```
x=-5:0.01:5;  
y=x.^2;  
plot(x,y)  
xlabel('Horizontal axis')  
ylabel('Vertical axis')  
title('The result of my first script is a Parabola')
```

After typing in the code you can run it, by clicking the "Run" icon on the "Editor" tab:

If the script wasn't saved previously, Matlab asks you to save the script in an *m* file.

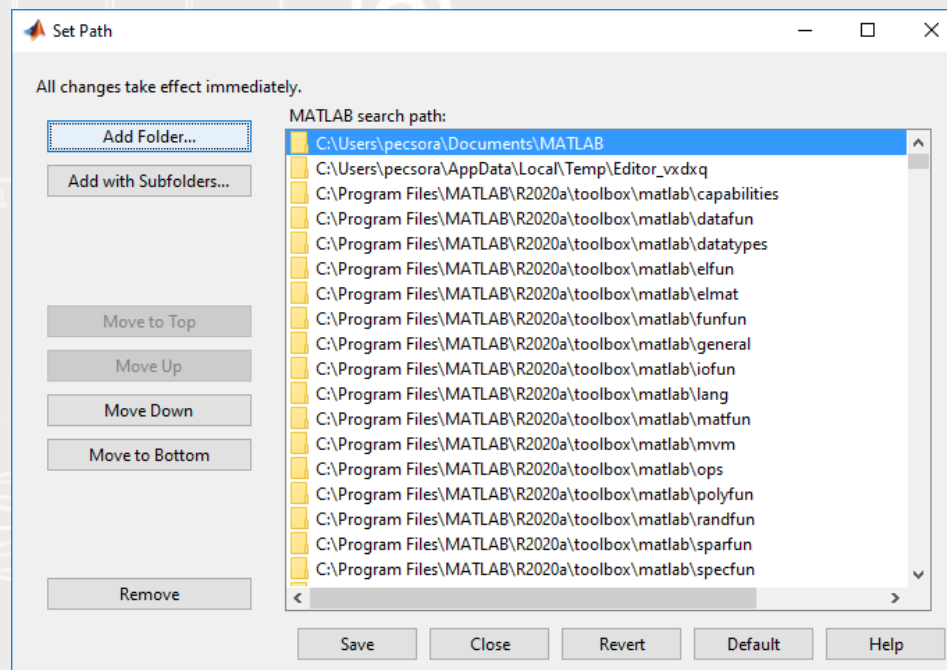
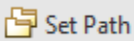
Let's name the script *parabola*.



My first scripts

If you save it in a folder which is in the "Path list", then you can run the script by typing *parabola* in the "Command Window".

If the folder is not in the "Path list", you can add your folder, by clicking the "Set Path" icon on the "Home" tab.



Keep in mind when using script files

1. The name of a script file must begin with a letter, and may include digits and the underscore character, up to 63 characters.
2. Do not give a script file the same name as a variable.
3. Do not give a script file the same name as a MATLAB command or function. You can check to see if a command, function or file name already exists by using the `exist` command.

COMMENTS

The comment symbol may be put anywhere in the line. MATLAB ignores everything to the right of the % symbol. For example,

```
% This is a comment.  
x = 2+3 % So is this.  
x =  
5
```

Note that the portion of the line before the % sign is executed to compute x.

Flow Control

if
for
while
break
...

Control Structures

If Statement Syntax

```
if (Condition_1)
    MATLAB Commands
elseif (Condition_2)
    MATLAB Commands
elseif (Condition_3)
    MATLAB Commands
else
    MATLAB Commands
end
```

Some Examples

```
if ((a>3) & (b==5))
    disp('Some MATLAB Commands');
end
```

```
if (a<3)
    disp('Some MATLAB Commands');
elseif (b==5)
    disp('Some other MATLAB Commands');
end
```

```
if (a<3)
    disp('Some MATLAB Commands');
else
    disp('Some other MATLAB Commands');
end
```

for loop

For loop syntax

```
for i=Index_Array
    MATLAB Commands
end
```

Some Examples

```
s=0;
for i=1:100
    s=s+i;
end
s
```

```
s=0;
for j=1:3:200
    s=s+i;
end
s
```

```
s=0;
for m=13:-0.2:-21
    s=s+m;
end
s
```

```
s=0;
for k=[0.1 0.3 -13 12 7 -9.3]
    s=s+k;
end
s
```

while loop

While Loop Syntax

while (condition)

MATLAB Commands

end

Example

```
a=10;  
b=5;  
while ((a>3) & (b==5))  
    disp('a is greater then 3 and b is equal to 5.');
```

a=a-1;
end

Exercise

E-learning exercise

Literature

- ❑ Ágnes Baran: Mathematics for engineers 1. (Laboratory slides)
- ❑ Sándor Baran: Probability theory and statistics
- ❑ Fazekas István: Valószínűségszámítás, Debreceni Egyetemi Kiadó, 2009
- ❑ Fazekas István: Bevezetés a matematikai statisztikába, Debreceni Egyetemi Kiadó, 2009
- ❑ Matlab examples:
<https://www.mathworks.com/help/examples.html>